03/9/26

Day 1 - SQL - Structured Query language

SQL foundations:

→ SQL is a standard programming language used to interact with relational database.

→ used to store, retrieve, update and delete data

→ modify / create database structures such as tables, views and indexes.

[Database is an organized collection of data that is stored electronically, handle large amount of information in various applications such as websites, business systems and e-commerce store].

Types of SQL commands.

→ DDL (data definition Language) - Defines the structure of the database

* create, alter, drop, truncate

→ DML (Data Manipulation Language) - controls / manage data stored in the database

* Select, Insert, Update, Delete

→ DCL (data Control Language) - controls access to the data

* Grant, Revoke

→ TCL (Transaction control Language) - Manages transactions in the database

* Commit, Rollback, Savepoint

→ DQL (Data Query Language) - Retrieves data from the database * Select

1. SELECT : Select is used to retrieve data faom one or more columns of a table

Syntax: Select column1, column2, ..
from table-name;

eg)
To retrive all columns :
↗ all
Select * from employees;

To retrive specfic columns:
Select employee-id, employee_name, salary
from employees;
(select decides what columns/data I want to see)

2) WHERE : used to filter individual rows based on a condition.

Syntax : Select columns
from table-name
where condition;

eg) Select * from employees
where salary > 50000;

Common operators used with where, or a condition

= equal, <> not equal, > greater than,
< lessthan, >= greater than or equal, <= Less than or equal

AND - Both conditions must be true

OR - at least one condition must be true

IN - used when checking multiple possible values

eg) where dept in ('IT', 'HR', 'Finance')

between - used to check a range.

eg) .where salary between 40000 and 70000;

LIKE - used for pattern matching.

eg) where name LIKE 'A%';

A% - word start with A.

%a - word ends with a.

%a% - contains a

> Key point
> where is used to filter rows before grouping

3) DISTINCT : removes duplicate values from the result

eg) select DISTINCT department from employees;

4) ORDER BY : used to sort the result

eg) select * from employees
ordering by salary ASC;

> keypoint: controls the order of the final result

ASC - ascending
DSC - desanding

can use both at a time
order by dept ASC, salary DESC

5) Aggregate functions : performs calculation on multiple rows and return a summarized result

some aggregate functions : count(), sum(), AVG(), MIN(), MAX()

i) count() - count rows in the table or a particular column, count don't count null values when used in a particular column.

eg) Select count(*) from employees;

Select count(employee_id) from employees; → ignore null values

ii) SUM() - calculates total

Select sum(salary) from employees;

iii) AVG() - calculates average of a column.

eg) Select avg (Salary) from employees;

iv) MIN() - finds minimum

eg) Select min (salary) from employees

v) MAX () - finds maximum of a column

eg) Select Max (salary) from employees;

combining all columns :

```
Select count(*) as employee_count
       Sum (salary) as total_salary,
       AVG (salary) as average_salary,
       MIN (salary) as minimum_salary
       MAX (salary) as maximum_salary
from employees;
```

as is used to give alias name

key point: Aggregate functions summarize multiple rows into meaningful business information

b) Group-by : groups rows having the same values.

commonly used with aggregate function

eg) Select dept, count (*) as employee_count
from employee
group by department;

dept	employee_count
I.T.	10
HR	5
Finance	7

key point: group by creates groups so aggregate calculation can be performed for each group

Where vs group by vs having

Table
↓
Where → filter individual rows
↓
group by → create groups
↓
Having → filter groups.
↓
Select
↓
orderby

> Where → before grouping
> Having → filter after grouping

eg) department having more than 5 employees

```
Select department, count(*) as employee_count
from employees
group by department
having count(*) > 5;
```

q) CASE : used to implement conditional logic in SQL
like (if, else if, else in pytho)

Syntax :

```
CASE
    when condition then result
    when condition then result
    else result
END
```

> In etl case is used in transformation covert raw data to business categories.

eg)

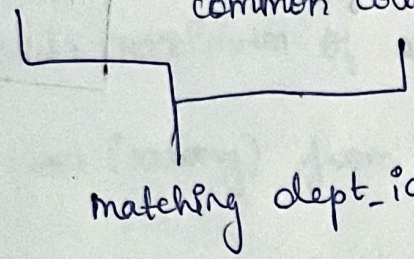
```
Select
    Employee_name,
    Salary,
    CASE
        When salary >= 70000 Then 'High'
        When salary >= 50000 Then 'medium'
    End as Salary_category
from employees;
```

04/9/26

10) Join - join is used combine two tables and retrieve data from them using a common column

i) inner join - Returns matching record from both tables

eg) Table 1 - employees Table 2 - department



ii) left join : return all join from the left table even if there is no match, and only the matching rows from right table

iii) Right join : Returns all rows from right table and matching rows from the left (*)

(prefer rewriting in left join for better pratice)

iv) full outer join : Return matching and non-matching records from both tables

left only + matching + right only

outer join merges columns vertical whereas union / union stack the rows vertically.
join can have different structure, union should have same structure.

SubQuery: Query written inside another Query

eg) finding employees earning more than the company's ave salary.

```
Select * from employees
where salary > ( select Avg (salary) from employees) ;
                 subquery
```

Union : union stack set vertically, include duplicates (slower performance)

union all : removes duplicates (faster).